System Architecture for Robotics Development

2026. 04. 17.

로보틱스 및 자율시스템 연구실

목차

1. Overview of Robot Development

2. Application Execution Process

2.1. Application Program

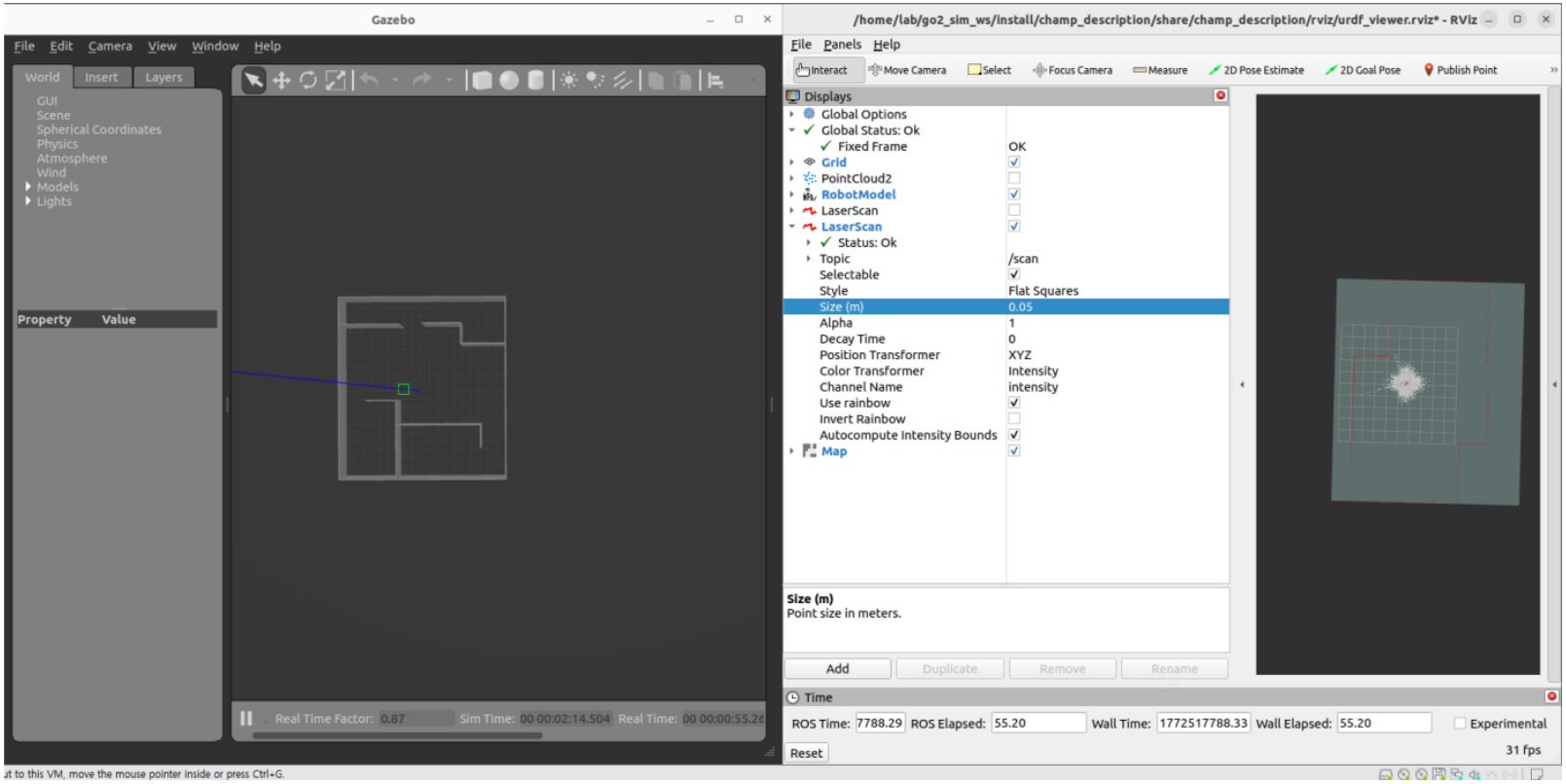
2.2. OS

2.3. ROS

1. Overview of Robot Development

Q. What is SLAM?

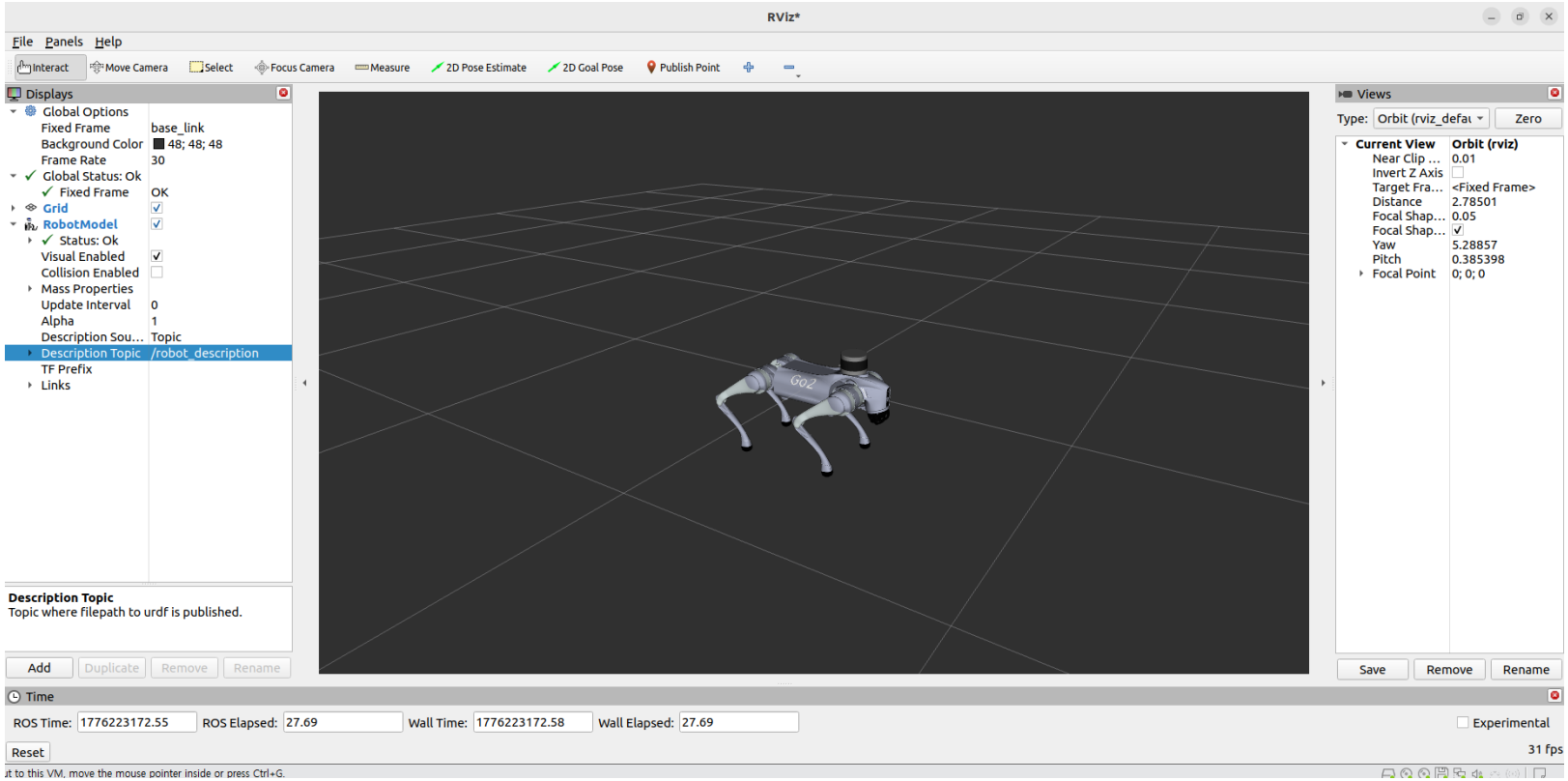
A. A technique that uses sensors to perceive the robot's current environment, generate a map, and simultaneously estimate the robot's position within the map



2. Overview of Robot Development

Q. What have we prepared for SLAM?

A. (1): Robot Visualization Using URDF



1. Overview of Robot Development

Q. How did you modify the URDF?

A. (1) Change the joint type from revolute to fixed

```
<joint
  name="FL_hip_joint"
  type="revolute">
  <origin
    xyz="0.1934 0.0465 0"
    rpy="0 0 0" />
  <parent
    link="base" />
  <child
    link="FL_hip" />
  <axis
    xyz="1 0 0" />
  <limit
    lower="-1.0472"
    upper="1.0472"
    effort="23.7"
    velocity="30.1" />
</joint>
```

```
<joint
  name="FL_hip_joint"
  type="fixed">
  <origin
    xyz="0.1934 0.0465 0"
    rpy="-1.0472 0 0" />
  <parent
    link="base" />
  <child
    link="FL_hip" />
</joint>
```

Q. Why are we changing the joint type from a revolute joint to a fixed joint?

A. If a revolute joint type exists, the robot's configuration becomes distorted unless a specific value is assigned to the joint.

1. Overview of Robot Development

Q. How did you modify the URDF?

A. (2) Rename base to base_link

```
<link
  name="base">
```

↕

```
<link
  name="base_link">
```

```yaml
slam_toolbox:
  ros__parameters:

    # Plugin params
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # ROS Parameters
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    use_map_saver: true
    mode: mapping #localization
```

Q. Why was base renamed to base_link?

A. For Nav2, the expected structure is map -> odom -> base_link -> base_footprint

1. Overview of Robot Development

Q. How did you modify the URDF?

A. (3) Add a base_footprint with base_link as its parent and add a joint

```xml
<link name="base_footprint"/>

<joint name="base_to_foot" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
  <!-- x y z r p y (rad) -->
  <origin xyz="0.0 0.0 0.30" rpy="0 0 0"/>
</joint>
```

```yaml
slam_toolbox:
  ros__parameters:

    # Plugin params
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # ROS Parameters
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    use_map_saver: true
    mode: mapping #localization
```

Q. Why was base renamed to base_link?

A. For Nav2, the expected structure is map -> odom -> base_link -> base_footprint

1. Overview of Robot Development

Q. How did you modify the URDF?

A. (4) Adding links and joints for LiDAR visualization

```
<link name="velodyne_base">
  <visual>
    <geometry>
      <mesh filename="package://velodyne_description/meshes/VLP16_base_1.dae"/>
    </geometry>
  </visual>
  <visual>
    <geometry>
      <mesh filename="package://velodyne_description/meshes/VLP16_base_2.dae"/>
    </geometry>
  </visual>
</link>
<joint name="base_to_velodyne_base" type="fixed">
  <parent link="base_link"/>
  <child link="velodyne_base"/>
  <origin xyz="0.18 0.00 0.09" rpy="0 0 0"/>
</joint>
```

```
<link name="velodyne">
  <visual>
    <origin xyz="0 0 -0.0377"/>
    <geometry>
      <mesh filename="package://velodyne_description/meshes/VLP16_scan.dae"/>
    </geometry>
  </visual>
</link>
<joint name="velodyne_base_to_scan" type="fixed">
  <parent link="velodyne_base"/>
  <child link="velodyne"/>
  <origin xyz="0 0 0.0377" rpy="0 0 0"/>
</joint>
```

Q. Why was base renamed to base_link?

A. This is to reflect the fact that LIDAR is mounted on the physical robot.

2. Overview of Robot Development

Q. What have we prepared for SLAM?

A. (2): odom topic publish

Publish

Topic: /utilidar/robot_odom
Type: nav_msgs/msg/Odometry

Subscription

Node

Publish

Topic: /odom
Type: nav_msgs/msg/Odometry

2. Overview of Robot Development

Q. How did you modify and publish Odom?

A. (1): Changed the topic name

Topic: /utilidar/robot_odom
Type: nav_msgs/msg/Odometry

↕

Topic: /odom
Type: nav_msgs/msg/Odometry

```yaml
bt_navigator:
  ros__parameters:
    use_sim_time: True
    global_frame: map
    robot_base_frame: base_link
    odom_topic: /odom
    bt_loop_duration: 10
    default_server_timeout: 20
    wait_for_service_timeout: 1000
    # 'default_nav_through_poses_bt_xml' and 'default_nav_to_pose_bt_xml' are use defaults:
    # nav2_bt_navigator/navigate_to_pose_w_replanning_and_recovery.xml
    # nav2_bt_navigator/navigate_through_poses_w_replanning_and_recovery.xml
    # They can be set here or via a RewrittenYaml remap from a parent launch file to Nav2.
    plugin_lib_names:
      - nav2_compute_path_to_pose_action_bt_node
      - nav2_compute_path_through_poses_action_bt_node
      - nav2_smooth_path_action_bt_node
      - nav2_follow_path_action_bt_node
      - nav2_spin_action_bt_node
      - nav2_wait_action_bt_node
      - nav2_assisted_teleop_action_bt_node
      - nav2_back_up_action_bt_node
      - nav2_drive_on_heading_bt_node
      - nav2_clear_costmap_service_bt_node
      - nav2_is_stuck_condition_bt_node
```

Q. Why did you change the topic name?

A. nav2 requires the /odom topic

2. Overview of Robot Development

Q. How did you modify and publish Odom?

A. (2): Changing child frame name

child_frame_id = robot_base

↕

child_frame_id = base_link

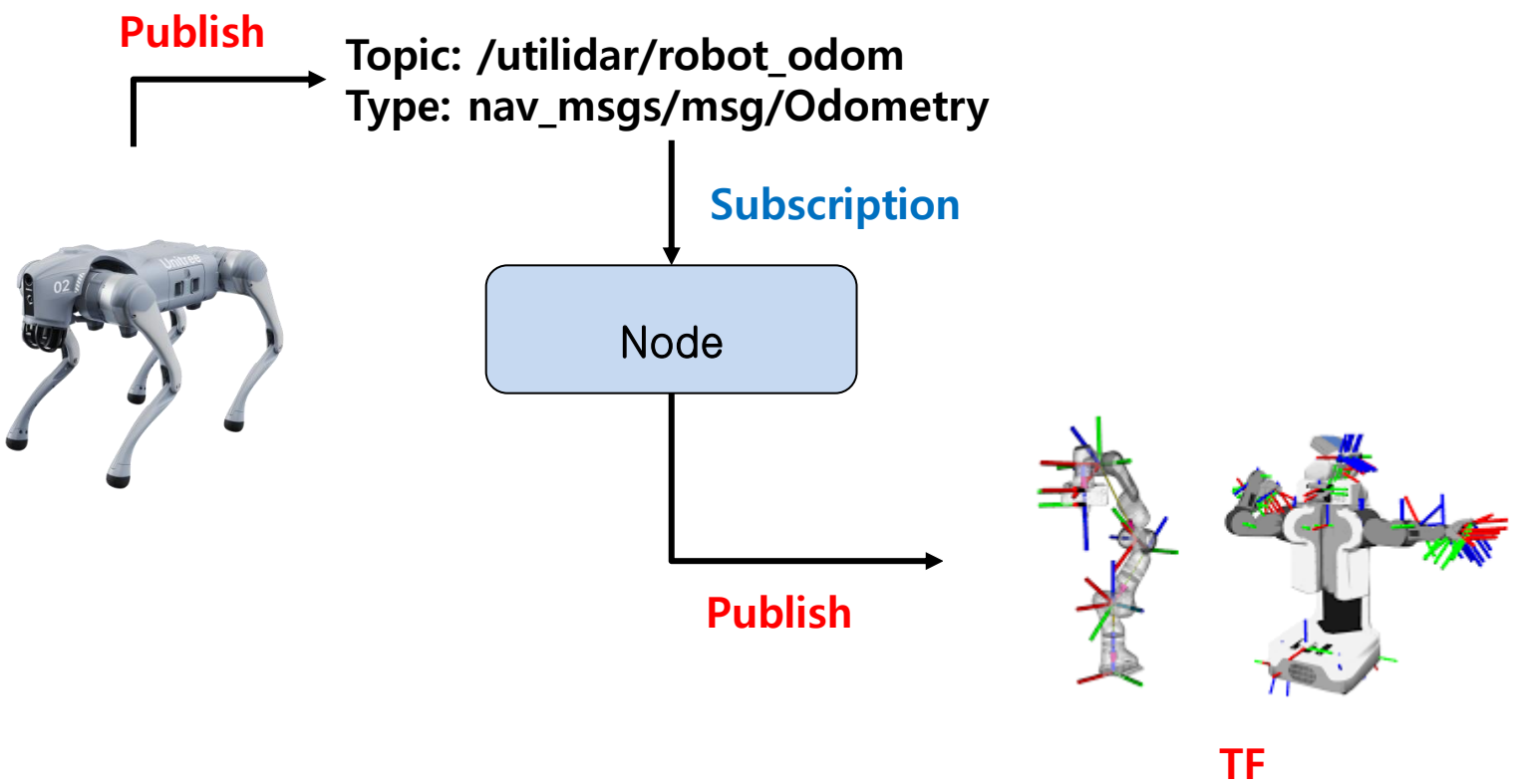
Q. Why did you change the name of the child frame?

A. nav2 expects a structure of map->odom->base_link->base_footprint, and this is done to align with the robot's center in the URDF.

2. Overview of Robot Development

Q. What have we prepared for SLAM?

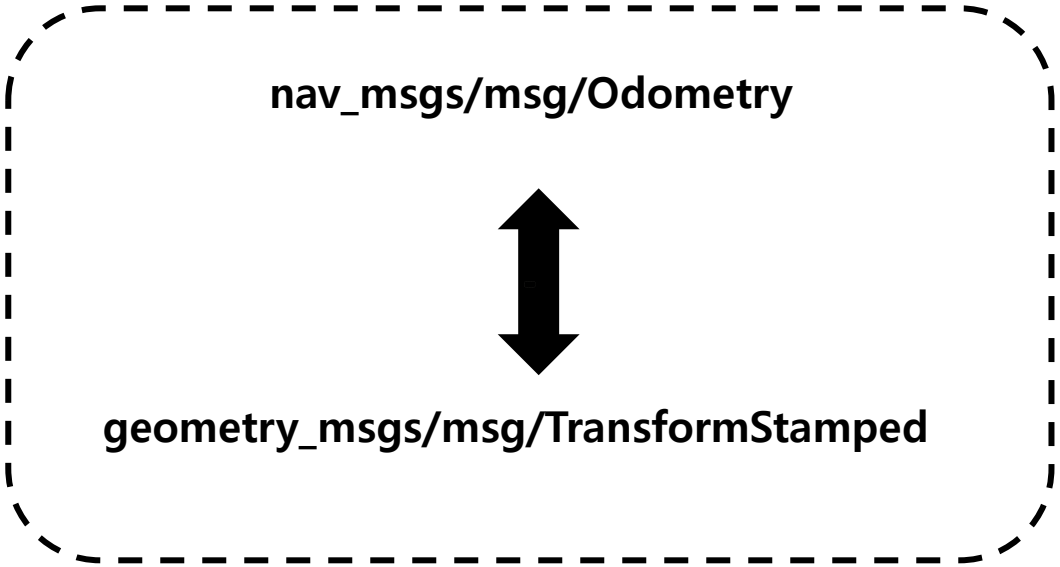
A. (3): TF publish



2. Overview of Robot Development

Q. How was the TF published?

A. (1): Change Data type



Q. Why did you change the data type?

A. Although Odom reports the movement and rotation between Odom and base_link, publishing TF is essential to actually reflect this, and TF must be published using a different data type.

2. Overview of Robot Development

Q. How was the TF published?

A. (2): Change child frame name

child_frame_id = robot_base

↕

child_frame_id = base_link

Q. Why did you change the name of the child frame?

A. nav2 expects a structure of map->odom->base_link->base_footprint, and this is done to align with the robot's center in the URDF.

2. Overview of Robot Development

Q. What have we prepared for SLAM?

A. (4): lidar topic name change and time synchronization

Publish → Topic: /scan
Type: sensor_msgs/msg/LaserScan

Publish → Topic: /scan_base
Type: sensor_msgs/msg/LaserScan

Subscription

```yaml
slam_toolbox:
  ros__parameters:

    # Plugin params
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # ROS Parameters
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    use_map_saver: true
    mode: mapping #localization
```

Node → Publish → Topic: /scan
Type: sensor_msgs/msg/LaserScan

2. Overview of Robot Development

Q. How was the scan data published?

A. (1): Changed the topic name published by the Velodyne driver from “scan” to “scan_base”

```
  sub_ = this->create_subscription<sensor_msgs::msg::PointCloud2>(
    "velodyne_points", rclcpp::QoS(10),
    std::bind(&VelodyneLaserScan::recvCallback, this, std::placeholders::_1));
  pub_ = this->create_publisher<sensor_msgs::msg::LaserScan>("scan", 10);
}
```

```
  sub_ = this->create_subscription<sensor_msgs::msg::PointCloud2>(
    "velodyne_points", rclcpp::QoS(10),
    std::bind(&VelodyneLaserScan::recvCallback, this, std::placeholders::_1));
  pub_ = this->create_publisher<sensor_msgs::msg::LaserScan>("scan_base", 10);
}
```

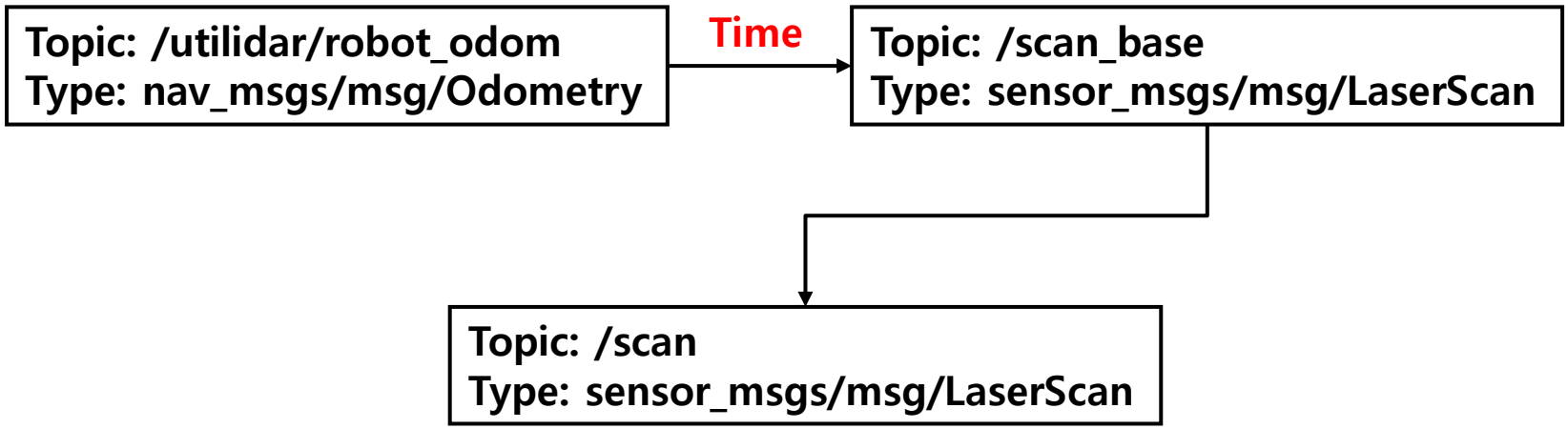
Q. Why did you change the data type?

A. nav2 requires the /scan topic. If the names of the topic being subscribed to and the topic being published to are the same, it may not function properly.

2. Overview of Robot Development

Q. How was the scan data published?

A. (2): Change the time of the /scan_base topic to match the time of /utilidar/robot_odom and publish it to /scan



Q. Why change the time?

A. In the case of NAV2 and SLAM, they will not function properly if the time is not synchronized